

Model Quality

Software Engineering for Intelligent Systems

JOÃO EDUARDO MONTANDON

How to measure



quality?

Quality comes in different flavors...

- Are required **features** implemented?
- Does it **operate** correctly under expected conditions?
- Does the implementation follow the intended **architecture**?

Quality comes in different flavors...

- Are required **features** implemented?
(Acceptance tests, test coverage)
- Does it **operate** correctly under expected conditions?
(Uptime period, Stress testing)
- Does the implementation follow the intended **architecture**?
(Static analysis, architectural reviews)

If we care about something, it must be detectable.

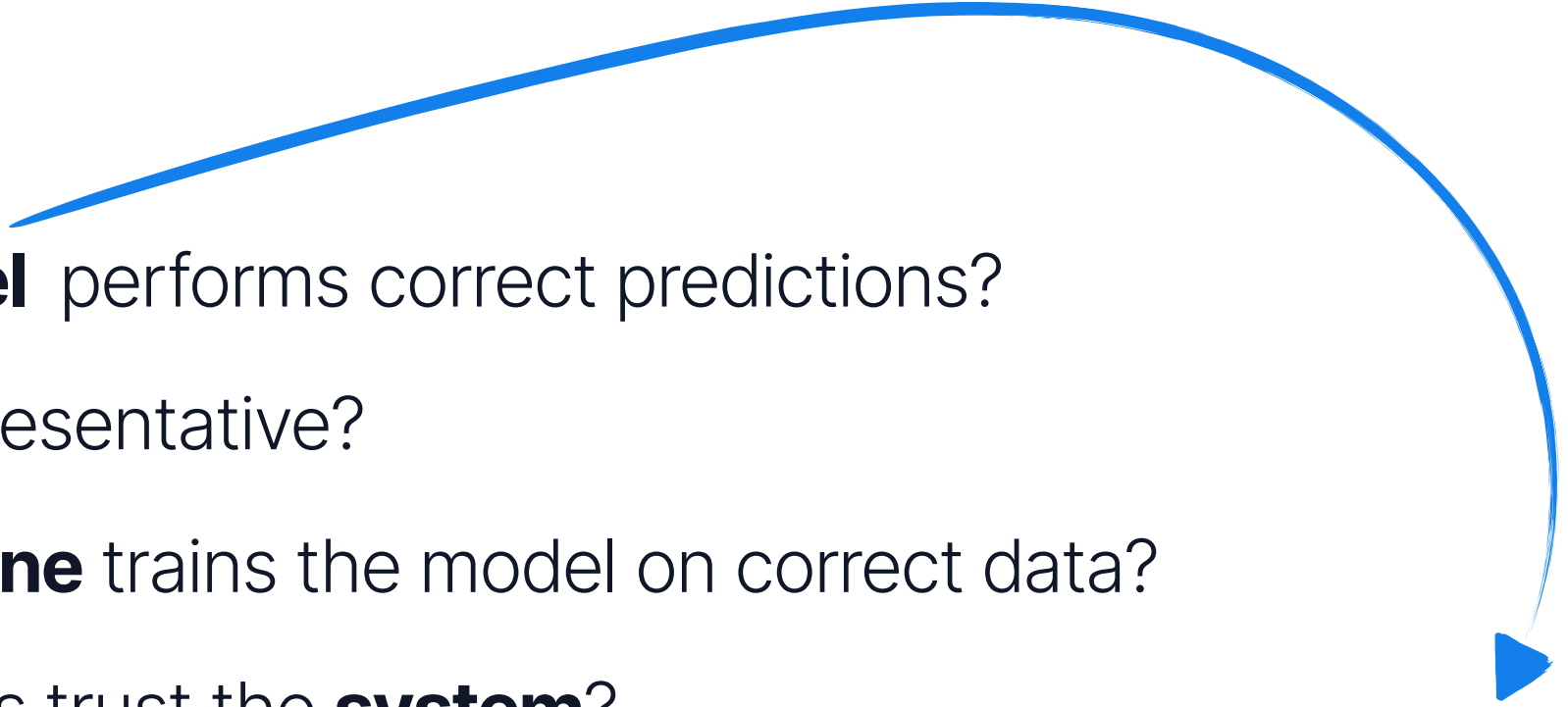
And we care about our models... right?

Intelligent Systems bring a lot of new concerns!

- Does the **model** performs correct predictions?
- Is the **data** representative?
- Does the **pipeline** trains the model on correct data?
- Do stakeholders trust the **system**?

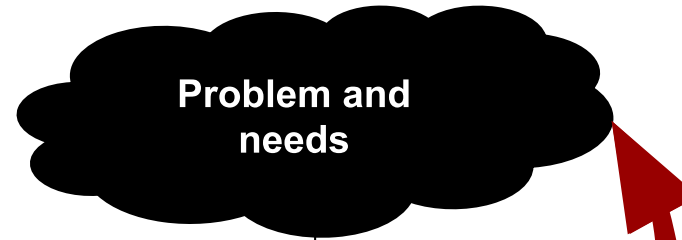
Intelligent Systems bring a lot of new concerns!

- Does the **model** performs correct predictions?
- Is the **data** representative?
- Does the **pipeline** trains the model on correct data?
- Do stakeholders trust the **system**?

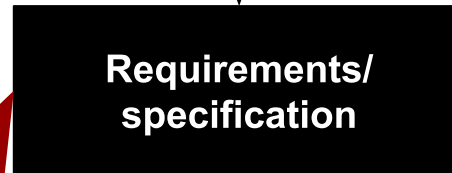


Our focus 

What is [model] correctness?



Requirements
engineering



Design
& coding

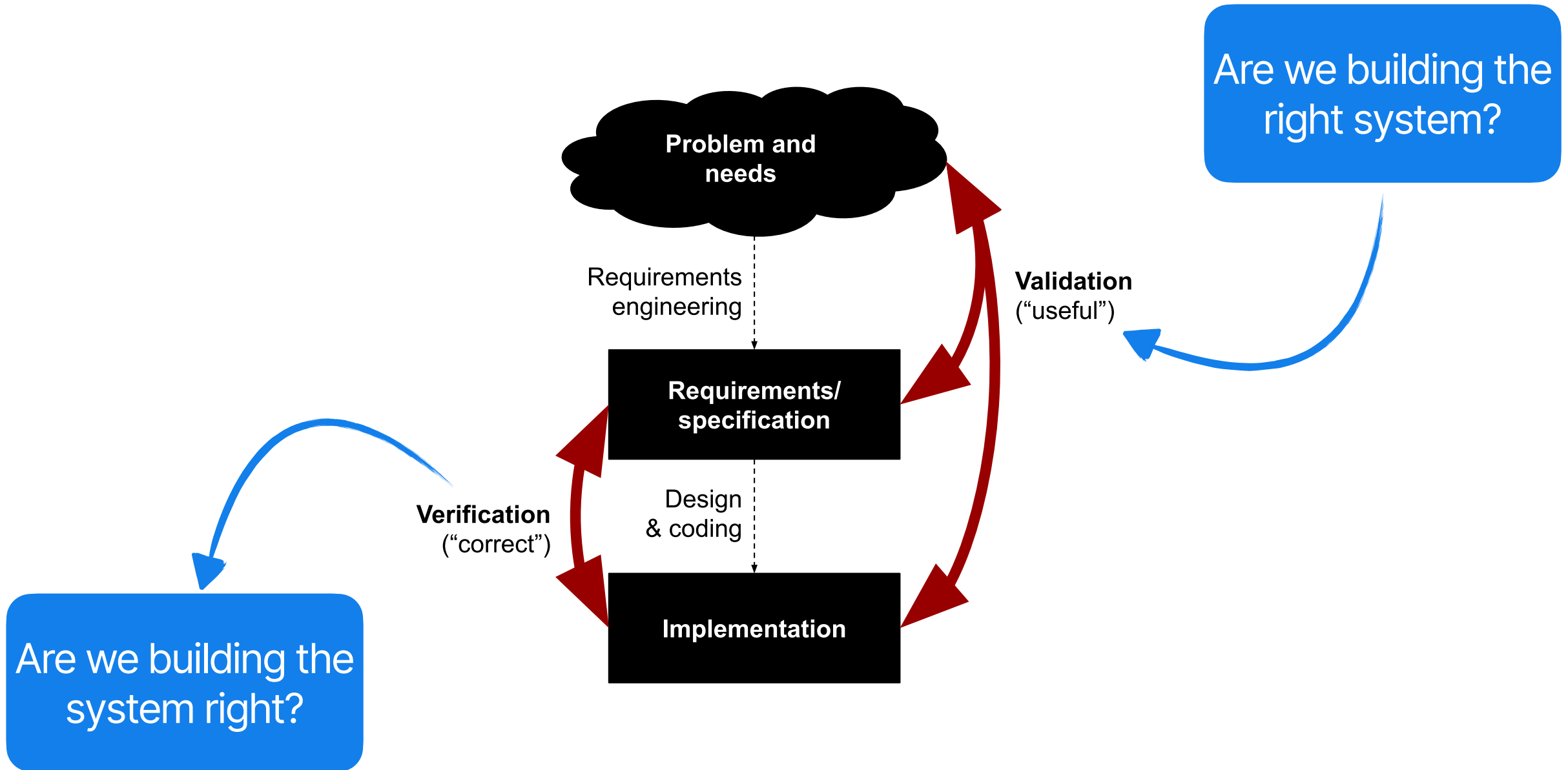


Validation
("useful")

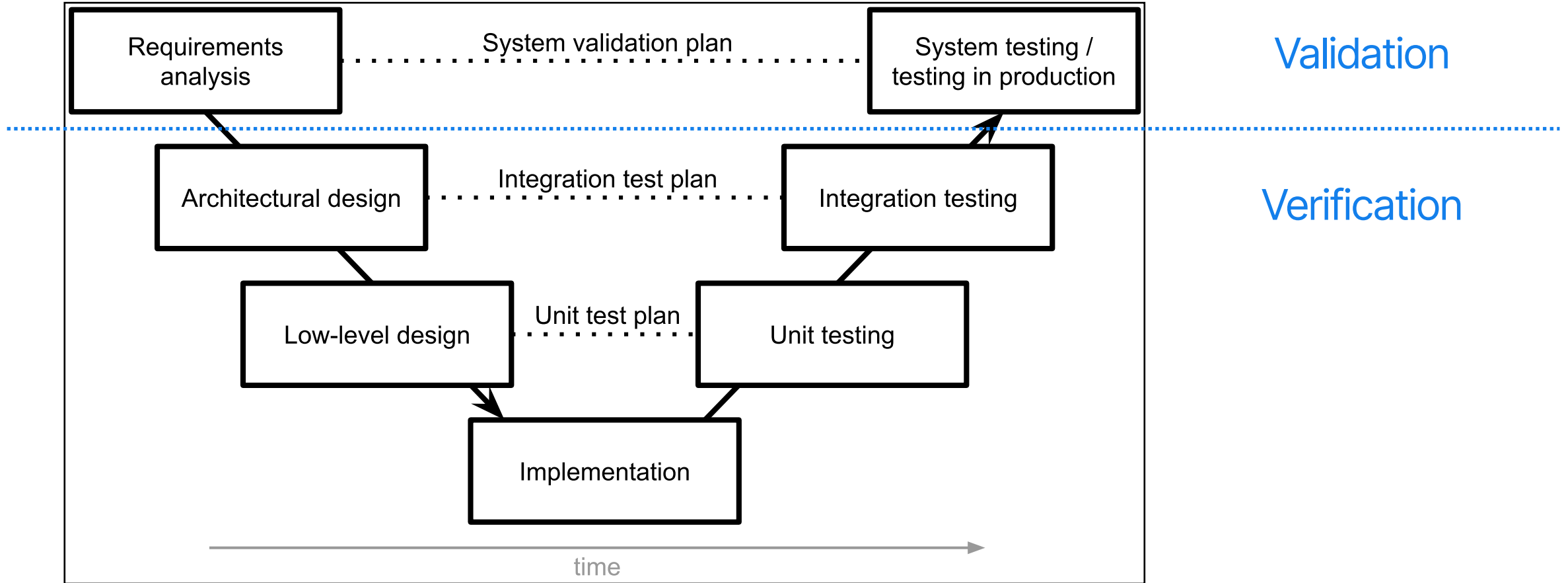
Verification
("correct")

Are we building the
right system?

Are we building the
system right?



Correct with regard to what?

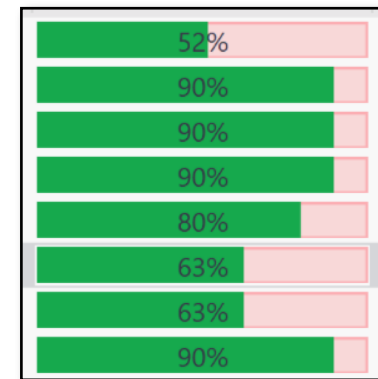


The verification lifecycle

```
def next_date(year, month, day):  
    """Spec: Given a year, a month (range 1-12), and a day (1-31),  
    the function returns the date of the following calendar day  
    in the Gregorian calendar as a triple of year, month, and day.  
    Throws InvalidInputException for inputs that are not valid dates.  
    """
```

```
def test_next_date_simple():  
    assertTrue(Date(2020,01,01).next() == Date(2020,01, 02))  
  
def test_next_date_end_of_31_day_month():  
    ...  
  
def test_next_date_end_of_30_day_month():  
    ...
```

90% are implemented
correctly



\$ pytest date.py

```
def hasCancer(image: Image, age: int, ...) → bool:
    """Determine whether the radiology image shows signs of cancer
    """
    """
```

No specification...

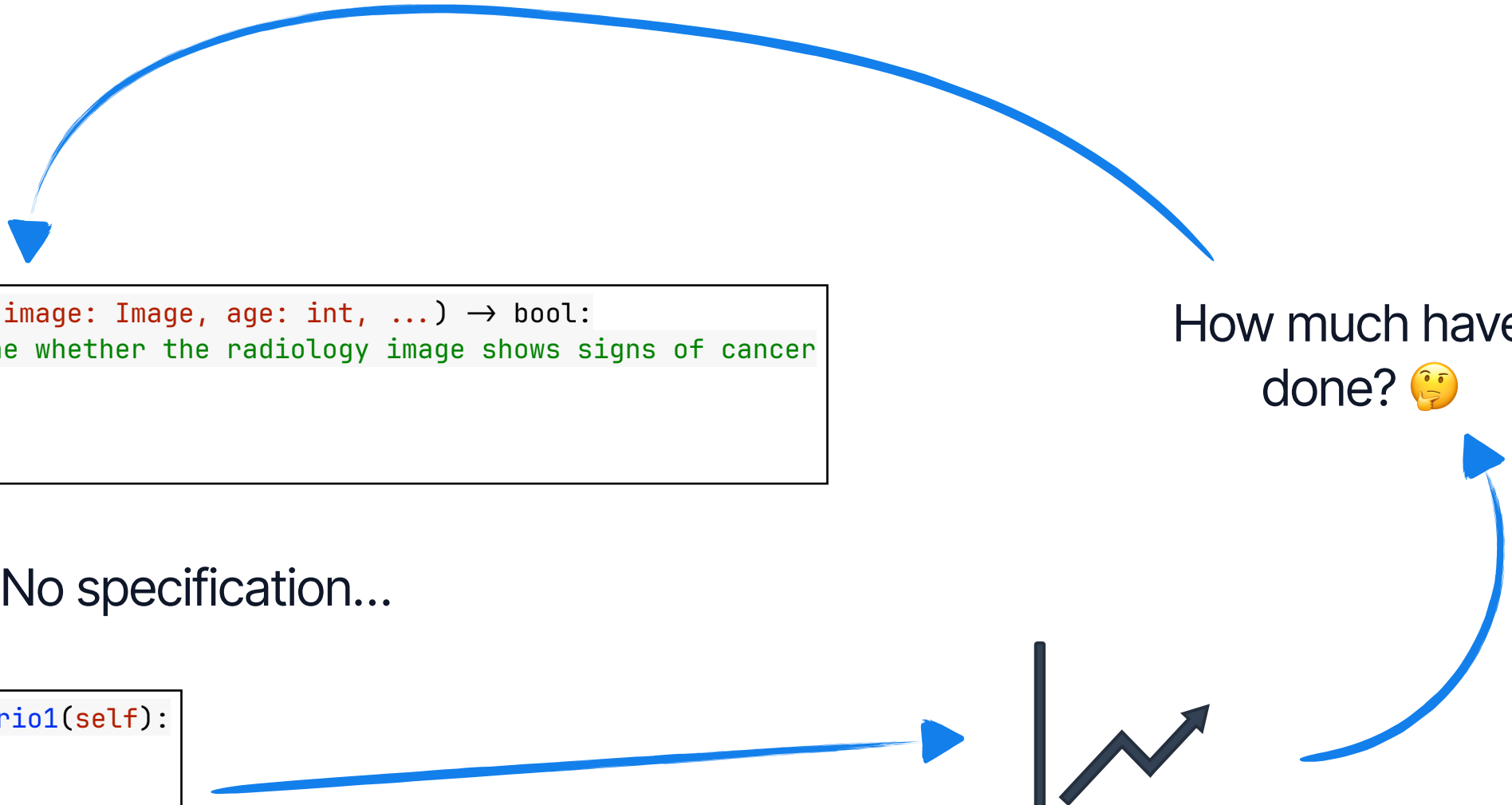
```
def test_scenario1(self):
    """

def test_scenario2(self):
    """
```

No tests....

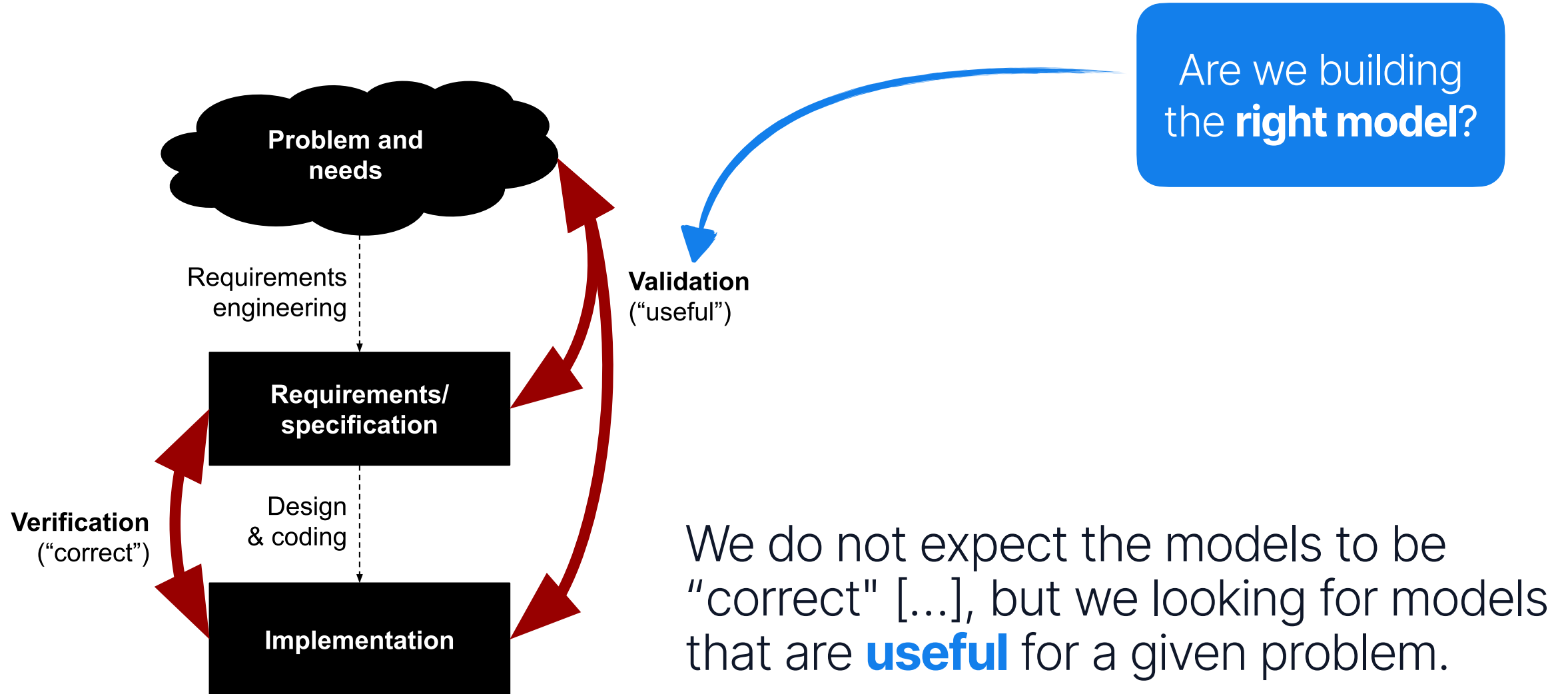


How much have I
done? 🤔

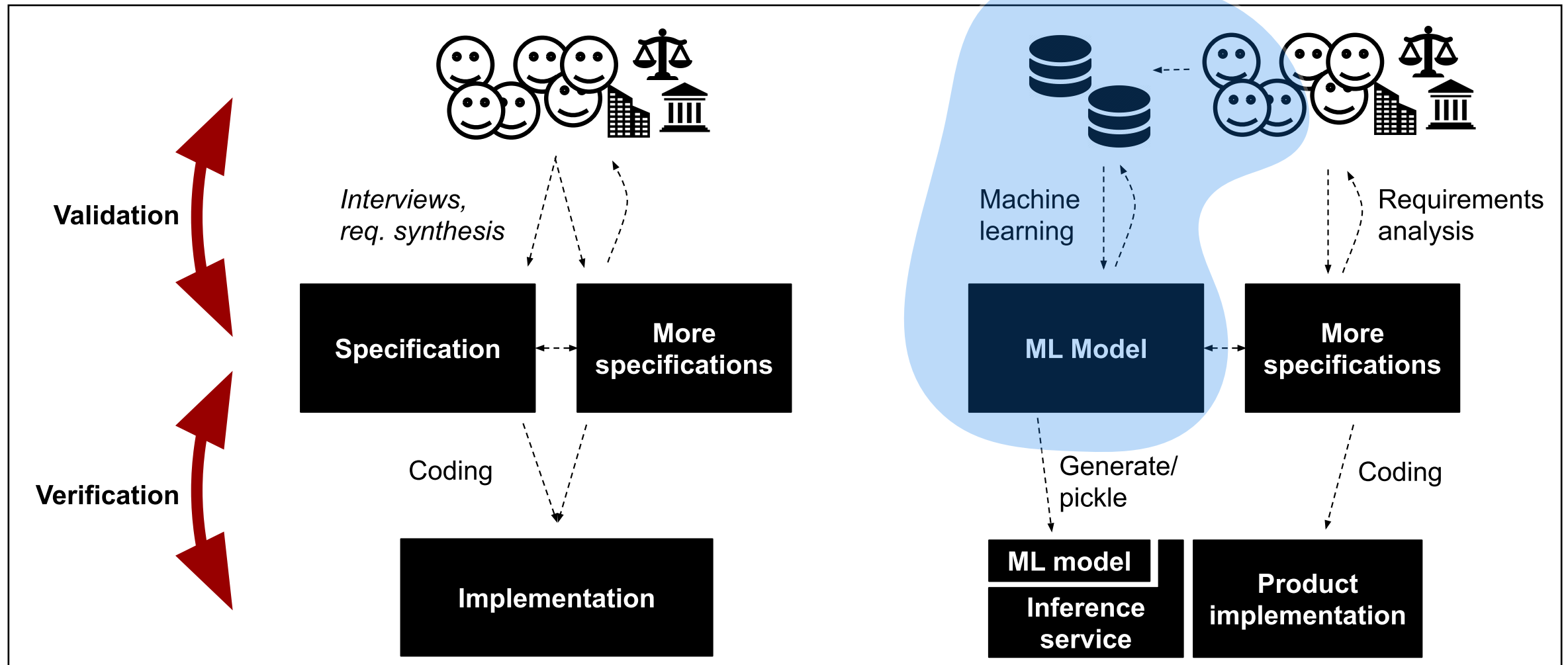


All models are wrong, but some are useful. So the question you need to ask is not 'Is the model true?' (It never is) but **'Is the model good enough for this particular application?'**"

— George Box



Model Quality



On Terminology

- Avoid "bug", since "bug" refers to requirement implemented incorrectly.
- Avoid "correctness", favor "fit" or "accuracy".
- In ML-world, *performance* $\equiv$ *accuracy*.
We will use accuracy.

Accuracy Measures

Model Accuracy

- The most common way to measure a model quality level.
- Model with higher accuracy is better than a model if a lower one.

There are tons of metrics

Our goal is to
understand **limitations**
and **pitfalls**

Classification	Accuracy, Precision, Recall, F1-score, ROC AUC, PR AUC, Log Loss, MCC
Regression	MSE, RMSE, MAE, MAPE, R^2 , Adjusted R^2
NLP	Perplexity, BLEU, ROUGE, METEOR, BERTScore, Exact Match, Token-level F1, Recall@k, MRR, NDCG

Remembering Confusion Matrix

		Actual	
		Positive	Negative
Predicted	Positive	TP	FP
	Negative	FN	TN

Accuracy

What is the proportion of correct classifications?

$$accuracy = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\begin{array}{|c|c|} \hline \text{blue} & \text{light blue} \\ \hline \text{light blue} & \text{blue} \\ \hline \end{array}}{\begin{array}{|c|c|} \hline \text{blue} & \text{blue} \\ \hline \text{blue} & \text{blue} \\ \hline \end{array}}$$

Accuracy

"Our ML model has achieved an accuracy of 0.97 for cancer detection". **How good is this?** 🤔

	Positive	Negative
Positive	??	??
Negative	??	??

Accuracy

$$\frac{4 + 9670}{4 + 325 + 1 + 9670} = 0.97$$

"Our ML model has achieved an accuracy of 0.97 for cancer detection". **How good is this?** 🤔

	Positive	Negative
Positive	4	325
Negative	1	9670



"Since all models are wrong, the scientist must be alert to
what is importantly wrong."

— George Box

Both FN and FP have costs, but they are not necessarily equal.
Which one to minimize?

Precision

What proportion of positive detections was actually correct?

$$\textit{precision} = \frac{\textit{correctly classified actual positives}}{\textit{everything classified as positive}} = \frac{TP}{TP + FP} = \frac{\begin{array}{|c|c|} \hline \text{blue} & \text{light blue} \\ \hline \text{light blue} & \text{light blue} \\ \hline \end{array}}{\begin{array}{|c|c|} \hline \text{blue} & \text{light blue} \\ \hline \text{blue} & \text{light blue} \\ \hline \end{array}}$$

Recall

What is the proportion of actual positives?

$$\text{recall} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN} = \frac{\begin{array}{|c|c|} \hline \text{blue} & \text{light blue} \\ \hline \text{light blue} & \text{light blue} \\ \hline \end{array}}{\begin{array}{|c|c|} \hline \text{blue} & \text{blue} \\ \hline \text{light blue} & \text{light blue} \\ \hline \end{array}}$$

$$prec = \frac{4}{4 + 325} = 0.01$$

$$rec = \frac{4}{4 + 1} = 0.80$$

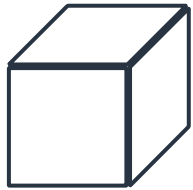
$$acc = \frac{4 + 9670}{4 + 325 + 1 + 9670} = 0.97$$

	Positive	Negative
Positive	4	325
Negative	1	9670

The importance of baselines

Which **improvement** is better?? 🤔

A:

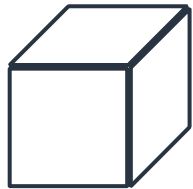


Acc = 50%



Acc = 75%

B:



Acc = 99.9%



Acc = 99.99%

The importance of baselines

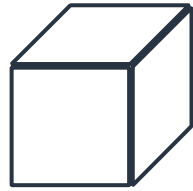
- Accuracy are meaningless if interpreted in isolation.
- There is no idea of progress.

$$\text{error reduction} = \frac{(1 - \text{baseline}) - (1 - \text{model})}{1 - \text{baseline}}$$

- **By how much** does the **new** model **reduce mistakes** compared to the **old** one?

$$\frac{(1 - 0.5) - (1 - 0.75)}{1 - 0.5} = \frac{0.25}{0.5} = 50\%$$

A:

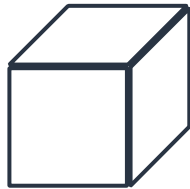


Acc = 50%



Acc = 75%

B:



Acc = 99.9%



Acc = 99.99%

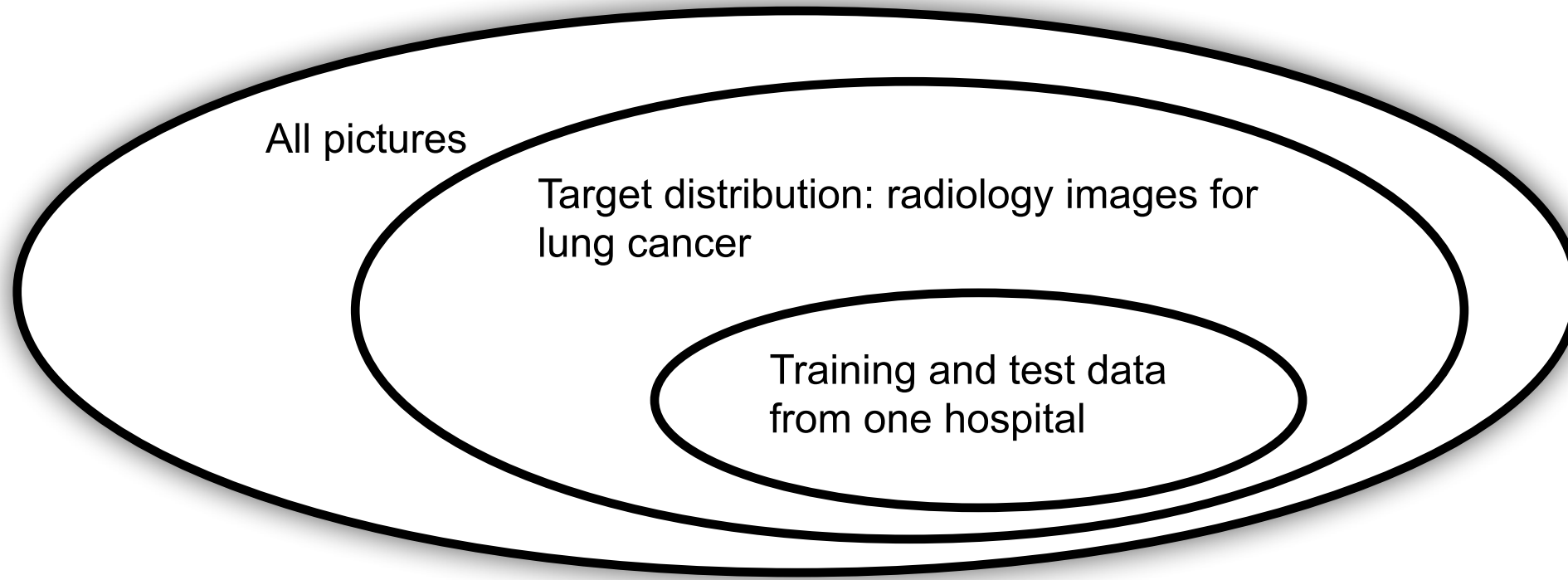
$$\frac{(1 - 0.99) - (1 - 0.9999)}{1 - 0.99} = \frac{0.0099}{0.01} = 99\%$$

Pitfalls

When measuring accuracy



P1. Using test data that are not representative



P2. Using misleading accuracy measures

- **Ignore the context** of your **application** when selecting a measure.
 - Accuracy for cancer detection, makes sense?
 - Report results without baseline?
- Observe **overall results** may overlook some nuances.
 - Cancer consistently fail at specific groups (black women).

P3. Data leakage

- Data can be shared between training and test/validation data in surprisingly ways.
 - Incorrect split
 - Data overlap
 - Unified preprocessing
- Every time a leak happens, the model can overfits.

P3. Example

Chenyang Yang, Rachel A Brower-Sinning, Grace Lewis, and Christian Kastner. 2023.
Data Leakage in Notebooks: Static Detection and Better Processes. ASE 2022.

```
1 import numpy as np
2 # generate random data
3 n_samples, n_features, n_classes = 200, 10000, 2
4 rng = np.random.RandomState(42)
5 X = rng.standard_normal((n_samples, n_features))
6 y = rng.choice(n_classes, n_samples)
7
8 # leak test data through feature selection
9 X_selected = SelectKBest(k=25).fit_transform(X, y)
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X_selected, y, random_state=42)
13 gbc = GradientBoostingClassifier(random_state=1)
14 gbc.fit(X_train, y_train)
15
16 y_pred = gbc.predict(X_test)
17 accuracy_score(y_test, y_pred)
18 # expected accuracy ~0.5; reported accuracy 0.76
```

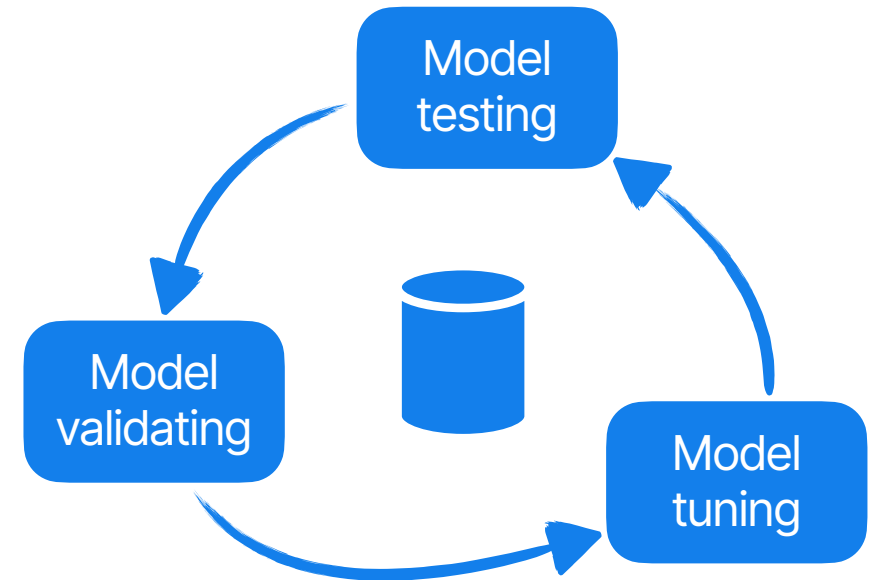
Train + test

Univariate feature selection works by selecting the best features based on univariate statistical tests. It can be seen as a preprocessing step to an estimator. Scikit-learn exposes feature selection routines as objects that implement the `transform` method:

- `SelectKBest` removes all but the k highest scoring features

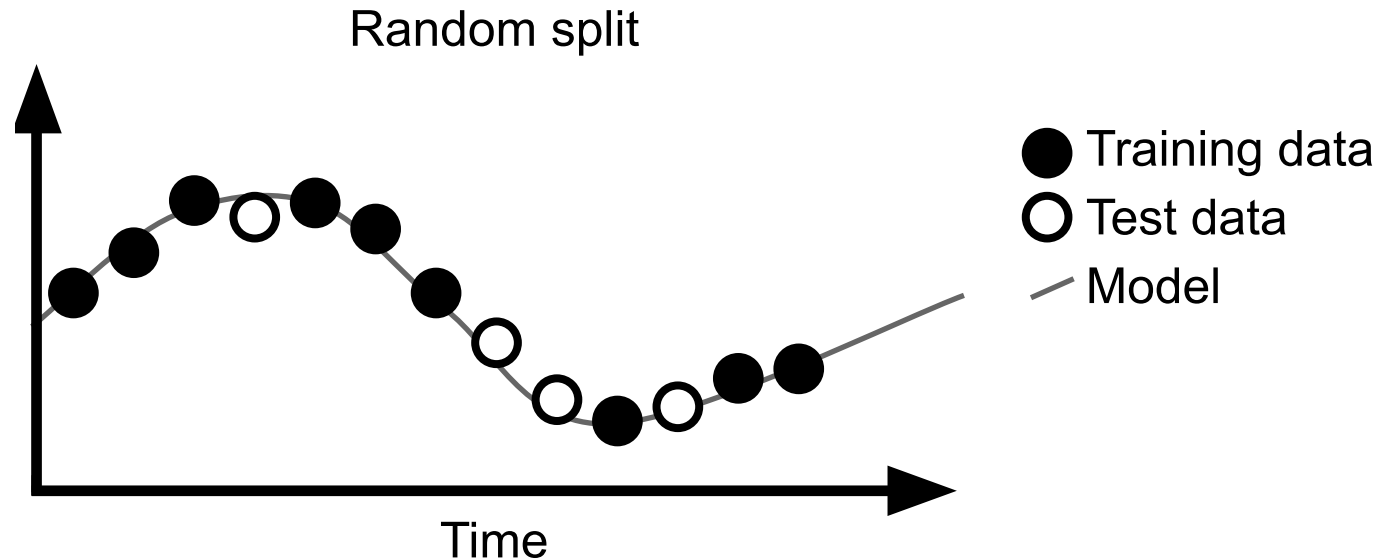
P4. Overfitting from repeated test data

- Happens when developers repeatedly improves the model on the same data.
- The model will increasingly be better only on that specific distribution.
 - a.k.a, overfitting
- Test data should be continuously refreshed.



P5. Data [in]dependence

- You are training a ML model for predicting stock prices.
- Your data contains the **stock price every hour**.
- You randomly split training and test data.

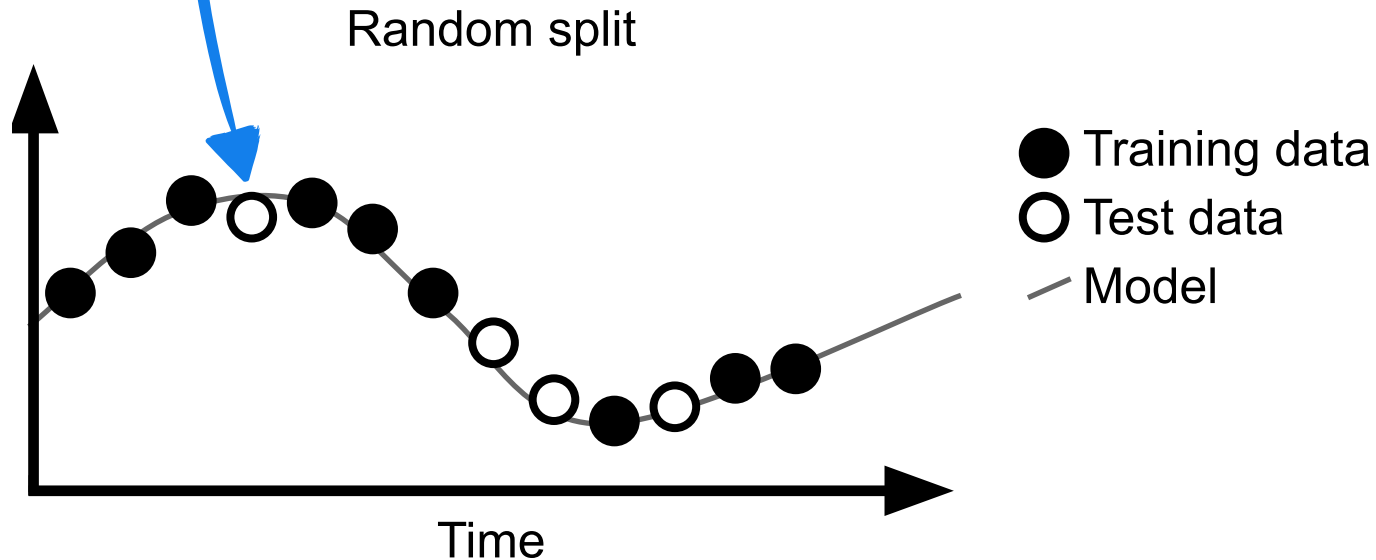


What is the problem here?? 🤔

P5. Data [in]dependence

Easier to predict a stock price if you have its price before and after!

This is an unrealistic scenario.

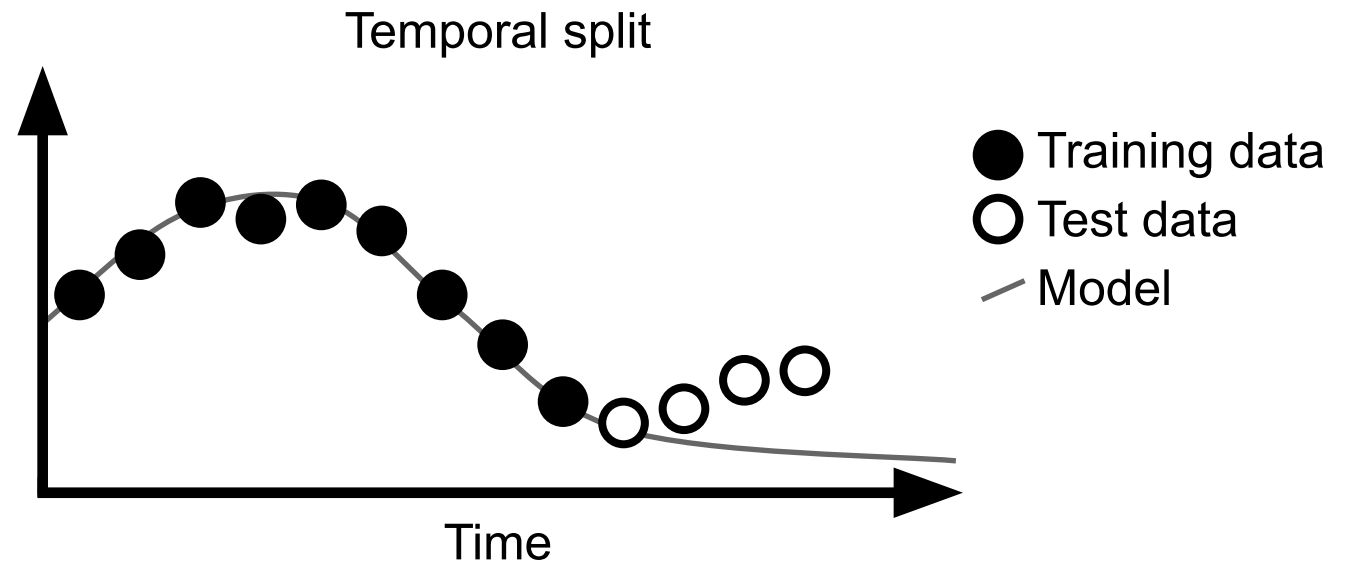


P5. Data [in]dependence

We should adopt a data **split technique** that keeps data points independent and realistic.

For example, group dependent data, and perform random sampling over the clusters.

Can you think on other data dependency scenario? 🤔



P6. Label Leakage

and the tank urban legend



P6. Label Leakage

- Happens when the model detects unrelated information that, somehow, leads to true label.
- The model does not learn the task, it detects the data leak, instead.
- This shortcut does not generalize in practice.

P6. Label Leakage

The classifier predicts "Wolf" if there is snow (or light background at the bottom), and "Husky" otherwise

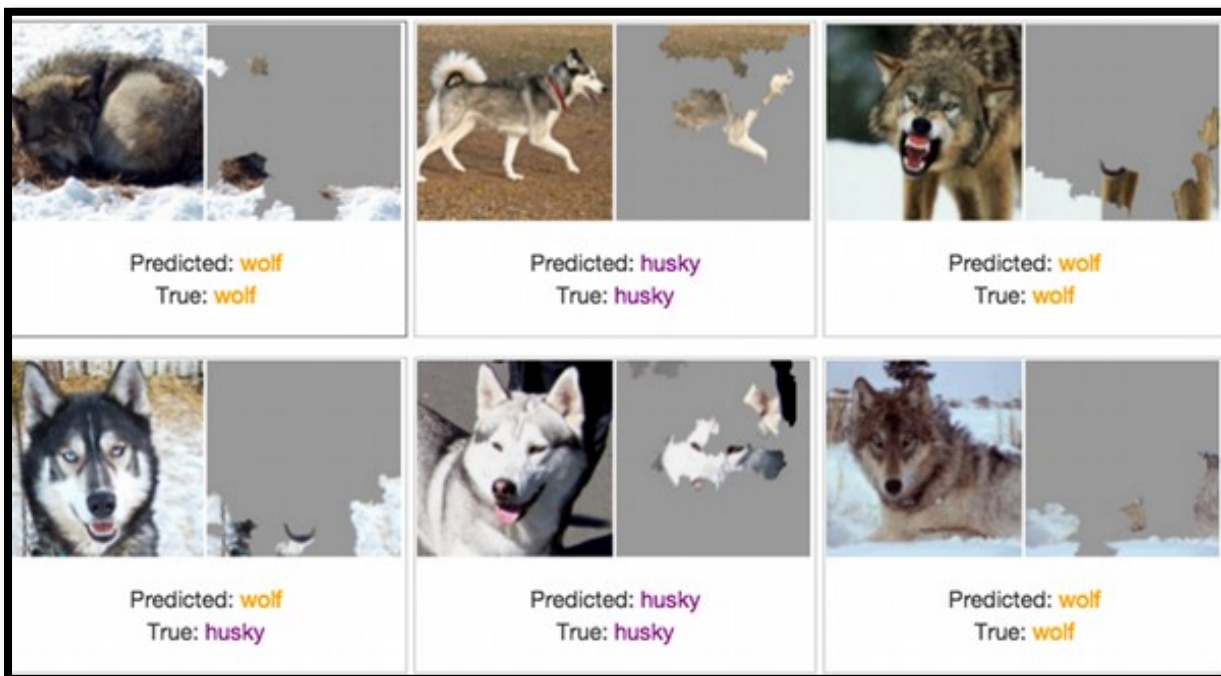
"Why Should I Trust You?" Explaining the Predictions of Any Classifier

Marco Tulio Ribeiro
University of Washington
Seattle, WA 98105, USA
marcotcr@cs.uw.edu

Sameer Singh
University of Washington
Seattle, WA 98105, USA
sameer@cs.uw.edu

Carlos Guestrin
University of Washington
Seattle, WA 98105, USA
guestrin@cs.uw.edu

<https://arxiv.org/pdf/1602.04938>



joao@dcc.ufmg.br